KITE

Минимальный язык программирования

Версия 1.0 · Полная документация

Быстрый · Читаемый · Без лишнего синтаксиса

Содержание

1. Введение
2. Установка и запуск
3. Синтаксис и структура
4. Типы данных
5. Переменные
6. Операторы
7. Строки
8. Списки
9. Словари (map)
10. Ветвление: when / orwhen / else
11. Циклы: loop while / loop for
12. Функции: def
13. Лямбды
14. Замыкания
15. Функции высшего порядка
16. Встроенные функции
17. Объекты через словари
18. Примеры программ
19. Обработка ошибок: do / err / end
20. Интерполяция строк ${}
21. Стандартные библиотеки: use math / rand / string / list / io / os
22. ООП: obj / extends / self / super
23. Инкапсуляция и полиморфизм

1. Введение

KITE — минималистичный язык программирования общего назначения, написанный на C. Цель: быть максимально простым в изучении, сохраняя достаточную мощь для реальных задач.

Основные принципы:

- **Минимализм** — всего 14 ключевых слов, никаких точек с запятой
- **Читаемость** — синтаксис читается как обычный текст
- **Функции первого класса** — функции передаются как значения
- **Динамическая типизация** — не нужно объявлять типы
- **Автоматическая память** — reference counting, утечек нет
- **Без зависимостей** — только libc + libm, работает везде

Пример — «Привет, мир»:

```
say("Привет, мир!")
```

Числа Фибоначчи:

```
# Числа Фибоначчи

def fib(n):

   when n <= 1: give n end

   give fib(n-1) + fib(n-2)

end


loop for i in range(10):

   say("fib(" .. i .. ") = " .. fib(i))

end
```

2. Установка и запуск

Требования: GCC (C11), make. Linux, macOS, Android (Termux).

Сборка из исходников:

```
tar -xzf kite-lang.tar.gz

cd kite-lang

make
```

Установка в Termux (Android):

```
make

cp kite $PREFIX/bin/kite

kite --version
```

Установка на Linux/macOS:

```
bash install.sh

# или вручную:

cp kite ~/.local/bin/kite

export PATH="$HOME/.local/bin:$PATH"
```

Режимы запуска:

Команда	Описание
`kite`	Запуск интерактивного REPL
`kite файл.kite`	Выполнить программу из файла
`kite --version`	Показать версию
`kite --help`	Показать справку
`kite --tokens файл.kite`	Показать токены (отладка)

Команды REPL:

Команда	Описание
`.help`	Быстрая справка по языку

Команда	Описание
`.env`	Показать все переменные окружения
`.quit`	Выйти из REPL

3. Синтаксис и структура

Отступы только для читаемости — они не несут синтаксического смысла. Блоки кода начинаются с двоеточия `:` и завершаются словом **end**. Каждая инструкция — одна строка. Комментарии начинаются с `#`.

```
# Это комментарий

set x = 42 # комментарий после кода


when x > 0:

    say("положительное") # блок кода

end
```

Все ключевые слова:

set	**def**	**give**	**end**
when	**orwhen**	**else**	**loop**
while	**for**	**in**	**break**
next	**and**	**or**	**not**
true	**false**	**nil**	

> **Важно:** в KITE нет точек с запятой. Каждая инструкция — одна строка. Блоки начинаются после двоеточия и заканчиваются ключевым словом **end**.

4. Типы данных

KITE — динамически типизированный язык. Тип определяется по значению автоматически. Функция **type**(x) возвращает имя типа строкой.

Тип	Примеры	Описание
num	42, 3.14, -7	Числа (double с плавающей точкой)
str	"привет", 'kite'	Строки в одинарных или двойных кавычках
bool	true, false	Логические значения
nil	nil	Отсутствие значения (аналог null)
list	[1, 2, 3]	Упорядоченный список любых значений
map	{x: 1, y: 2}	Словарь: строковые ключи, любые значения
fn	def f(): ... end	Функция (def или лямбда)
builtin	say, len, map...	Встроенная функция интерпретатора

```
say(type(42)) # num

say(type("hello")) # str

say(type(true)) # bool

say(type(nil)) # nil

say(type([1,2,3])) # list

say(type({a: 1})) # map

say(type(say)) # builtin
```

5. Переменные

Переменные объявляются ключевым словом **set**. Переприсваивание — просто знак =. Область видимости лексическая.

```
set x = 42
set name = "Алиса"
set pi = 3.14159
set ok = true
set nothing = nil
set nums = [1, 2, 3]
set point = {x: 10, y: 20}

# Переприсваивание
x = x + 1 # теперь x = 43
x = 100
```

Составные операторы присваивания:

Оператор	Эквивалент	Пример
+=	x = x + n	x += 5
-=	x = x - n	x -= 3
*=	x = x * n	x *= 2
/=	x = x / n	x /= 4

```
set score = 100
score += 50 # 150
score -= 20 # 130
score *= 2 # 260
score /= 4 # 65
say(score) # 65
```

6. Операторы

Арифметические операторы:

Оператор	Название	Пример	Результат
`+`	Сложение	5 + 3	8
`-`	Вычитание	10 - 4	6
`*`	Умножение	3 * 7	21
`/`	Деление	15 / 4	3.75
`%`	Остаток от деления	10 % 3	1
`^`	Возведение в степень	2 ^ 10	1024
`..`	Конкатенация строк	"a" .. "b"	"ab"
`*`	Повтор строки	"ha" * 3	"hahaha"

Операторы сравнения:

Оператор	Описание	Пример
`==`	Равенство	x == 42
`!=`	Неравенство	x != 0
`<`	Строго меньше	a < b
`>`	Строго больше	a > b
`<=`	Меньше или равно	a <= 10
`>=`	Больше или равно	score >= 100

Логические операторы:

Оператор	Описание	Пример
`and`	Оба условия истинны	x > 0 and x < 10
`or`	Хотя бы одно истинно	x == 0 or x == 1
`not`	Инверсия условия	not ok

Приоритет операторов (от высшего к низшему):
унарный минус → `^` → `* / %` → `+ -` `..` → `== != < > <= >=` → **`not`** → **`and`** → **`or`** → `=`

7. Строки

Строки — неизменяемые последовательности символов. Поддерживаются одинарные и двойные кавычки. Escape-последовательности: \n (перенос), \t (таб), \\ (бэкслэш), \" и \' (кавычки).

```
set s1 = "Привет, мир!"

set s2 = 'тоже строка'

set s3 = "Строка с переносом\nстроки"

set s4 = "Вкладка:\tтекст"

# Операции

say("Hello" + " " + "World") # Hello World

say("hi" .. "!") # hi!

say("ha" * 3) # hahaha

say(len("kite")) # 4

say("hello"[0]) # h

say("hello"[-1]) # o
```

Встроенные функции для строк:

Функция	Описание	Пример
`upcase(s)`	В верхний регистр	upcase("hello") → "HELLO"
`downcase(s)`	В нижний регистр	downcase("HI") → "hi"
`trim(s)`	Убрать пробелы по краям	trim(" ok ") → "ok"
`len(s)`	Длина строки	len("kite") → 4
`split(s, sep)`	Разбить на список	split("a,b", ",") → ["a","b"]
`join(list, sep)`	Собрать из списка	join(["a","b"], "-") → "a-b"
`slice(s, n, m)`	Подстрока [n:m]	slice("hello", 1, 4) → "ell"
`replace(s, a, b)`	Замена подстроки	replace("aaa","a","b") → "bbb"
`index_of(s, sub)`	Позиция подстроки	index_of("hello","ll") → 2

Функция	Описание	Пример
`has(s, sub)`	Содержит ли подстроку	has("hello","ell") → true
`str(v)`	Любое значение в строку	str(42) → "42"

```
set s = " Привет, KITE! "
set s = trim(s) # "Привет, KITE!"
set s = downcase(s) # "привет, kite!"
say(len(s)) # 13
say(slice(s, 0, 6)) # "привет"

set words = split("один,два,три", ",")
say(join(words, " + ")) # "один + два + три"

# Итерация по символам
loop for ch in "kite":
   say(ch)
end
```

8. Списки (list)

Список — упорядоченная коллекция значений любых типов. Индексация с 0. Отрицательные индексы: –1 это последний элемент. Списки изменяемы.

```
set a = [10, 20, 30, 40, 50]

say(a[0]) # 10 — первый
say(a[-1]) # 50 — последний
say(a[-2]) # 40 — предпоследний
say(len(a)) # 5

a[0] = 99 # изменить элемент
say(a) # [99, 20, 30, 40, 50]
```

Функция	Описание	Пример
`push(list, v)`	Добавить в конец (изменяет список)	push(a, 60)
`pop(list)`	Удалить и вернуть последний	pop(a) → 50
`len(list)`	Количество элементов	len(a) → 5
`slice(list,n,m)`	Подсписок [n:m]	slice(a,1,3)
`reverse(list)`	Перевёрнутая копия	reverse([1,2,3])
`concat(a, b)`	Объединить два списка	concat([1],[2,3])
`sort(list)`	Отсортированная копия	sort([3,1,2])
`sort(list, fn)`	Сортировка с компаратором	sort(a,(x,y)->x-y)
`has(list, v)`	Содержит ли элемент	has(a,30) → true
`map(list, fn)`	Преобразовать каждый элемент	map(a,x->x*2)
`filter(list, fn)`	Оставить подходящие	filter(a,x->x>20)
`reduce(list,fn,i)`	Свернуть в одно значение	reduce(a,(a,b)->a+b,0)

```
set nums = [5, 2, 8, 1, 9, 3]
```

```
say(sort(nums)) # [1, 2, 3, 5, 8, 9]

say(sort(nums, (a,b) -> b - a)) # [9, 8, 5, 3, 2, 1]

say(min(nums)) # 1

say(max(nums)) # 9

say(slice(nums, 1, 4)) # [2, 8, 1]

say(concat([1,2,3], [4,5,6])) # [1, 2, 3, 4, 5, 6]
```

9. Словари (map)

Словарь — коллекция пар ключ–значение. Ключи — строки. Значения — любого типа. Порядок ключей сохраняется.

```
set person = {
   name: "Алиса",
   age: 30,
   city: "Москва"
}

# Два способа доступа:
say(person["name"]) # через строку
say(person.age) # через точку

# Добавить / изменить:
person["job"] = "Инженер"
person["age"] = 31
```

Функция	Описание	Пример
keys(map)	Список всех ключей	keys(p) → ["name","age",...]
vals(map)	Список всех значений	vals(p) → ["Алиса", 30, ...]
len(map)	Количество ключей	len(p) → 3
has(map,k)	Проверить наличие ключа	has(p,"name") → true

```
# Вложенные структуры
set data = {
   user: {name: "Боб", scores: [95, 87, 92]},
   active: true
}
say(data["user"]["name"]) # Боб
```

```
say(data["user"]["scores"][0]) # 95

# Итерация по ключам
loop for k in keys(data["user"]):
    say(k .. " = " .. str(data["user"][k]))
end
```

10. Ветвление: when / orwhen / else

Условная конструкция: **when** (аналог if), **orwhen** (аналог else if), **else**. Завершается **end**.

```
when x > 100:
    say("очень большое")
orwhen x > 50:
    say("большое")
orwhen x > 0:
    say("положительное")
orwhen x == 0:
    say("ноль")
else:
    say("отрицательное")
end
```

Однострочная запись:

```
when n <= 1: give n end

when x > 0: say("да") else: say("нет") end
```

Сложные условия:

```
when x > 0 and x < 100:
    say("от 1 до 99")
end

when name == "admin" or name == "root":
    say("суперпользователь")
```

```
end

when not ok:
    say("что-то пошло не так")
end
```

11. Циклы: loop while / loop for

Все циклы начинаются с **loop**. Цикл с условием: **loop while**. Цикл по коллекции: **loop for**. Управление: **break** (выход), **next** (следующая итерация).

loop while — цикл с условием:

```
set i = 0
loop while i < 5:
    say(i)
    i += 1
end
# выведет: 0 1 2 3 4
```

loop for — цикл по коллекции:

```
loop for fruit in ["яблоко", "банан", "вишня"]:
    say("фрукт:", fruit)
end

# По символам строки
loop for ch in "KITE":
    say(ch)
end

# По ключам словаря
set m = {a: 1, b: 2}
loop for key in keys(m):
    say(key, "=", m[key])
end
```

Функция range() для числовых диапазонов:

Вызов	Генерирует список
`range(5)`	[0, 1, 2, 3, 4]
`range(1, 6)`	[1, 2, 3, 4, 5]
`range(0, 10, 2)`	[0, 2, 4, 6, 8]
`range(5, 0, -1)`	[5, 4, 3, 2, 1]

```
loop for i in range(1, 6):
   say(i, "^2 =", i ^ 2)
end
```

break и next:

```
# break — выйти из цикла
loop for i in range(10):
   when i == 5: break end
   say(i)
end

# next — пропустить итерацию
loop for i in range(10):
   when i % 2 == 0: next end
   say(i) # только нечётные: 1 3 5 7 9
end
```

12. Функции: def

Функции объявляются через **def**. Возврат значения — **give** (аналог return). Без **give** функция возвращает nil. Функции — полноправные значения: хранятся в переменных, передаются как аргументы.

```
def greet(name):
   give "Привет, " .. name .. "!"
end

say(greet("Мир")) # Привет, Мир!
```

Рекурсия:

```
def factorial(n):
   when n <= 1: give 1 end
   give n * factorial(n - 1)
end

loop for i in range(1, 8):
   say(i .. "! = " .. factorial(i))
end
```

Функции как значения:

```
def double(x):
   give x * 2
end

def apply(fn, x):
   give fn(x)
end

say(apply(double, 5)) # 10
```

```
set f = double

say(f(7)) # 14
```

13. Лямбды (анонимные функции)

Лямбды создаются оператором `->`. Тело — одно выражение (автоматически возвращается).

```
# Один параметр
set square = x -> x * x
set negate = x -> -x

say(square(5)) # 25
say(negate(3)) # -3

# Несколько параметров
set add = (a, b) -> a + b
set mul = (a, b) -> a * b

say(add(3, 4)) # 7

# Передача как аргумент
set nums = range(1, 6)
say(map(nums, x -> x * x)) # [1, 4, 9, 16, 25]
say(filter(nums, x -> x % 2 == 0)) # [2, 4]

# Фабрика функций
def adder(n):
   give x -> x + n
end

set add10 = adder(10)
say(add10(5)) # 15
```

14. Замыкания (closures)

Замыкание — функция, которая запоминает переменные из области видимости, где была создана. Все функции в KITE автоматически образуют замыкание.

```
def make_counter(start):
   set count = start
   def inc():
     count += 1
     give count
   end
   give inc
end

set c1 = make_counter(0)
set c2 = make_counter(100)

say(c1()) # 1
say(c1()) # 2
say(c1()) # 3
say(c2()) # 101 — независимый счётчик!
say(c1()) # 4
```

Аккумулятор:

```
def make_accumulator():
   set total = 0
   def add(n):
     total += n
     give total
```

```
    end
    give add
end

set acc = make_accumulator()
say(acc(10)) # 10
say(acc(5)) # 15
say(acc(25)) # 40
```

Каждый вызов `make_counter()` создаёт **новое независимое замыкание** со своей копией переменной `count`. Счётчики `c1` и `c2` не влияют друг на друга.

15. Функции высшего порядка

Функции высшего порядка принимают функции как аргументы или возвращают их. Три главные встроенные ФВП: map, filter, reduce.

map — преобразовать каждый элемент:

```
set nums = [1, 2, 3, 4, 5]

say(map(nums, x -> x * x)) # [1, 4, 9, 16, 25]

say(map(nums, x -> x * 2)) # [2, 4, 6, 8, 10]

say(map(nums, x -> "№" .. x)) # ["№1", "№2", ...]
```

filter — оставить подходящие:

```
set nums = range(1, 11)

say(filter(nums, x -> x % 2 == 0)) # [2, 4, 6, 8, 10]

say(filter(nums, x -> x > 7)) # [8, 9, 10]
```

reduce — свернуть в одно значение:

```
set nums = [1, 2, 3, 4, 5]

say(reduce(nums, (a, b) -> a + b, 0)) # 15

say(reduce(nums, (a, b) -> a * b, 1)) # 120
```

Цепочка операций:

```
# Сумма квадратов чисел > 9 из диапазона 1..10

set result = reduce(

   filter(

     map(range(1, 11), x -> x * x),

     x -> x > 9

   ),

   (a, b) -> a + b,

   0
```

```
)

say(result) # 371
```

Сортировка с компаратором:

```
set words = ["банан", "яблоко", "вишня", "дата"]

say(sort(words)) # алфавит

say(sort(words, (a, b) -> len(a) - len(b))) # по длине
```

16. Встроенные функции

Ввод / Вывод:

Функция	Описание
`say(...)`	Вывести значения через пробел + перенос строки
`input(подсказка)`	Читать строку от пользователя

Типы и преобразования:

Функция	Описание	Пример
`type(v)`	Имя типа значения	type(42) → "num"
`str(v)`	В строку	str(3.14) → "3.14"
`num(v)`	В число	num("42") → 42

Математика:

Функция / Константа	Описание	Пример
`sqrt(x)`	Квадратный корень	sqrt(144) → 12
`abs(x)`	Абсолютное значение	abs(-7) → 7
`floor(x)`	Округление вниз	floor(3.9) → 3
`ceil(x)`	Округление вверх	ceil(3.1) → 4
`round(x)`	Округление	round(3.5) → 4
`min(...)`	Минимум значений/списка	min(3,1,4) → 1
`max(...)`	Максимум значений/списка	max(3,1,4) → 4
`sin(x)`	Синус (радианы)	sin(PI/2) → 1
`cos(x)`	Косинус (радианы)	cos(0) → 1
`log(x)`	Натуральный логарифм	log(1) → 0
`PI`	Число π ≈ 3.14159	PI
`TAU`	2π ≈ 6.28318	TAU

Коллекции:

Функция	Описание
`len(v)`	Длина строки / списка / словаря

Функция	Описание
`range(n)`	Список [0, 1, ..., n-1]
`range(a, b)`	Список [a, a+1, ..., b-1]
`range(a, b, s)`	Список a..b с шагом s
`push(list, v)`	Добавить v в конец (изменяет список)
`pop(list)`	Удалить и вернуть последний элемент
`has(col, v)`	Проверить наличие элемента / ключа / подстроки
`slice(v, n, m)`	Подсписок или подстрока [n:m]
`reverse(v)`	Перевёрнутая копия
`concat(a, b)`	Объединить два списка
`sort(list)`	Отсортированная копия (по возрастанию)
`sort(list, fn)`	Отсортированная копия с компаратором
`keys(map)`	Список ключей словаря
`vals(map)`	Список значений словаря
`map(list, fn)`	Преобразовать каждый элемент
`filter(list, fn)`	Оставить элементы, для которых fn = true
`reduce(list,fn,i)`	Свернуть список в одно значение

17. Объекты через словари

KITE не имеет встроенных классов. Объекты имитируются словарями, а методы — функциями, принимающими объект первым аргументом.

Точка на плоскости:

```
def make_point(x, y):
    give {x: x, y: y}
end

def point_str(p):
    give "(" .. p["x"] .. ", " .. p["y"] .. ")"
end

def point_dist(p, q):
    set dx = q["x"] - p["x"]
    set dy = q["y"] - p["y"]
    give sqrt(dx*dx + dy*dy)
end

set a = make_point(0, 0)
set b = make_point(3, 4)
say(point_str(a)) # (0, 0)
say(point_str(b)) # (3, 4)
say(point_dist(a, b)) # 5
```

Стек (Stack):

```
def make_stack():
    give {items: [], size: 0}
end
```

```
def stack_push(s, v):
   push(s["items"], v)
   s["size"] += 1
end
def stack_pop(s):
   s["size"] -= 1
   give pop(s["items"])
end
def stack_peek(s):
   give s["items"][s["size"] - 1]
end

set st = make_stack()
stack_push(st, 10)
stack_push(st, 20)
stack_push(st, 30)
say("вершина:", stack_peek(st)) # 30
say("извлечено:", stack_pop(st)) # 30
```

18. Примеры программ

FizzBuzz:

```
loop for i in range(1, 31):
    when i % 15 == 0: say("FizzBuzz")
    orwhen i % 3 == 0: say("Fizz")
    orwhen i % 5 == 0: say("Buzz")
    else: say(i)
    end
end
```

Быстрая сортировка (Quicksort):

```
def qsort(arr):
    when len(arr) <= 1: give arr end
    set pivot = arr[floor(len(arr) / 2)]
    set left = filter(arr, x -> x < pivot)
    set mid = filter(arr, x -> x == pivot)
    set right = filter(arr, x -> x > pivot)
    give concat(concat(qsort(left), mid), qsort(right))
end

say(qsort([3, 6, 8, 10, 1, 2, 1])) # [1,1,2,3,6,8,10]
```

Бинарный поиск:

```
def bin_search(arr, target, lo, hi):
    when lo > hi: give -1 end
    set mid = floor((lo + hi) / 2)
    when arr[mid] == target: give mid end
```

```
    when arr[mid] < target:
      give bin_search(arr, target, mid+1, hi)
    end
    give bin_search(arr, target, lo, mid-1)
  end

  set s = [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]
  say(bin_search(s, 23, 0, len(s)-1)) # 5
  say(bin_search(s, 99, 0, len(s)-1)) # -1
```

Подсчёт слов:

```
  def word_count(text):
    set words = filter(split(text, " "), w -> len(w) > 0)
    set counts = {}
    loop for word in words:
      set w = downcase(word)
      when has(counts, w): counts[w] += 1
      else: counts[w] = 1
      end
    end
    give counts
  end

  set wc = word_count("kite is fast kite is simple")
  loop for k in keys(wc):
    say(k .. ": " .. wc[k])
  end
```

19. Обработка ошибок: do / err / end

Блок **do** выполняет код и перехватывает ошибки. После **err** указывается тип ошибки — если он совпадает, выполняется блок обработчика. **err** без типа ловит любую ошибку. Блоков **err** может быть несколько. Завершается **end**.

Базовый синтаксис:

```
do:
    # код который может упасть
err ТипОшибки:
    # обработчик
end
```

Примеры:

```
# Поймать конкретную ошибку
do:
    say(1 / 0)
err ZeroDivisionError:
    say("деление на ноль!")
end

# Несколько обработчиков
do:
    set a = [1, 2, 3]
    say(a[99])
err IndexError:
    say("индекс вне диапазона")
err TypeError:
```

```
    say("неверный тип")
err:
    say("другая ошибка: ${err_msg}")
end

# Поймать любую ошибку
do:
    say(undefined_var)
err:
    say("ошибка: ${err_msg}")
    say("тип: ${err_type}")
end

# Работа с файлами
use io
do:
    set data = file_read("config.txt")
    say(data)
err IOError:
    say("файл не найден, создаём...")
    file_create("config.txt")
end
```

Переменные внутри блока err:

Переменная	Содержит
err_msg	Текст ошибки
err_type	Тип ошибки строкой (например "ZeroDivisionError")

Все типы ошибок:

Тип	Когда возникает
`ZeroDivisionError`	Деление на ноль
`NameError`	Обращение к неопределённой переменной
`IndexError`	Индекс за пределами списка
`TypeError`	Неверный тип значения для операции
`IOError`	Ошибка при работе с файлами
`ImportError`	Неизвестная библиотека в use
`RuntimeError`	Прочие ошибки времени выполнения

> **Важно:** если ни один обработчик не совпал — ошибка продолжает распространяться выше. Программа продолжает работу нормально после того как ошибка поймана.

20. Интерполяция строк

Строки поддерживают встроенную интерполяцию через синтаксис `${{...}}`. Внутри скобок можно писать любое выражение — переменную, вызов функции, арифметику.

```
set name = "Алиса"
set age = 30
set nums = [1, 2, 3]

say("Привет, ${name}!")
say("Через 5 лет будет ${age + 5}")
say("Список: ${nums}")
say("Длина: ${len(nums)}")
say("Тип: ${type(age)}")
say("ИМЯ: ${upcase(name)}")

# Несколько в одной строке
set x = 10
set y = 20
say("${x} + ${y} = ${x + y}")
say("${x} * ${y} = ${x * y}")

# В функциях
def greet(who, n):
   give "Привет, ${who}! Тебе ${n} лет."
end
say(greet("Боб", 25))
```

> **Отличие от конкатенации:** вместо `'Привет, ' .. name .. '!'` пишем `'Привет, ${name}!'` — короче и читаемее. Экранирование: `\${` выведет буквально `${`.

21. Стандартные библиотеки

Библиотеки подключаются командой **use** в начале файла. После этого все функции библиотеки становятся доступны. Можно подключать несколько библиотек.

```
use math

use rand

use string

use list

use io

use os
```

use math

Расширенная математика: тригонометрия, логарифмы, утилиты.

Функция / Константа	Описание	Пример
`tan(x)`	Тангенс	tan(PI/4) → 1
`asin(x)`	Арксинус	asin(1) → PI/2
`acos(x)`	Арккосинус	acos(1) → 0
`atan(x)`	Арктангенс	atan(1) → 0.785
`atan2(y,x)`	Арктангенс двух арг.	atan2(1,1) → 0.785
`exp(x)`	e в степени x	exp(1) → 2.718
`log2(x)`	Логарифм по основ. 2	log2(8) → 3
`log10(x)`	Логарифм по основ. 10	log10(100) → 2
`pow(x, y)`	x в степени y	pow(2,10) → 1024
`hypot(x, y)`	Гипотенуза	hypot(3,4) → 5
`trunc(x)`	Отрезать дробь	trunc(3.9) → 3
`sign(x)`	Знак числа (-1/0/1)	sign(-5) → -1
`clamp(v,a,b)`	Ограничить диапазон	clamp(15,0,10) → 10
`lerp(a,b,t)`	Линейная интерполяция	lerp(0,100,0.3) → 30

Функция / Константа	Описание	Пример
`is_nan(x)`	Проверить NaN	is_nan(0) → false
`is_inf(x)`	Проверить Inf	is_inf(INF) → true
`E`	Число e ≈ 2.71828	E
`INF`	Бесконечность	INF

use rand

Генерация случайных чисел. Без rand_seed — каждый раз разные числа.

Функция	Описание	Пример
`rand()`	Случайное число 0.0..1.0	rand() → 0.734
`rand(n)`	Случайное целое 0..n-1	rand(10) → 7
`rand(a, b)`	Случайное целое a..b-1	rand(5,10) → 8
`rand_seed(n)`	Фиксировать генератор	rand_seed(42)
`rand_choice(list)`	Случайный элемент списка	rand_choice(['a','b','c'])
`rand_shuffle(list)`	Перемешать список (копия)	rand_shuffle([1,2,3,4,5])

```
use rand

rand_seed(42) # фиксируем для воспроизводимости

say(rand()) # 0.0334

say(rand(100)) # 0..99

say(rand_choice(["красный","синий","зелёный"]))

say(rand_shuffle([1,2,3,4,5]))
```

use string

Дополнительные функции для работы со строками.

Функция	Описание	Пример
`str_repeat(s, n)`	Повторить строку n раз	str_repeat("ab",3) → "ababab"
`str_pad_left(s,w,c)`	Дополнить слева до ширины w	str_pad_left("42",6,"0") → "000042"

Функция	Описание	Пример
`str_pad_right(s,w,c)`	Дополнить справа до ширины w	str_pad_right("hi",5,".") → "hi..."
`str_count(s, sub)`	Сколько раз sub встречается	str_count("banana","a") → 3
`str_rev(s)`	Перевернуть строку	str_rev("hello") → "olleh"
`str_is_num(s)`	Является ли строка числом	str_is_num("3.14") → true
`str_lines(s)`	Разбить по переносам строки	str_lines("a\nb\nc")
`str_chars(s)`	Список символов строки	str_chars("kite")

use list

Утилиты для работы со списками.

Функция	Описание	Пример
`list_sum(list)`	Сумма всех элементов	list_sum([1,2,3]) → 6
`list_avg(list)`	Среднее арифметическое	list_avg([1,2,3]) → 2
`list_uniq(list)`	Убрать дубликаты	list_uniq([1,1,2,3]) → [1,2,3]
`list_flat(list)`	Развернуть вложенные списки	list_flat([[1,2],[3]]) → [1,2,3]
`list_zip(a, b)`	Объединить два списка парами	list_zip([1,2],[a,b])
`list_count(list, v)`	Посчитать вхождения v	list_count([1,1,2],1) → 2
`list_take(list, n)`	Взять первые n элементов	list_take([1,2,3,4],2)
`list_drop(list, n)`	Пропустить первые n	list_drop([1,2,3,4],2)
`list_any(list, fn)`	Хотя бы один подходит	list_any(nums, x->x>5)
`list_all(list, fn)`	Все подходят	list_all(nums, x->x>0)
`list_find(list, fn)`	Первый подходящий элемент	list_find(nums, x->x>7)
`list_group(list, fn)`	Сгруппировать в словарь	list_group(nums, x->x%2)

```
use list

set ns = [3, 1, 4, 1, 5, 9, 2, 6]
```

```
say(list_sum(ns)) # 31

say(list_avg(ns)) # 3.875

say(list_uniq(ns)) # [3, 1, 4, 5, 9, 2, 6]

say(list_zip([1,2,3], ["a","b","c"])) # [[1,"a"],[2,"b"],[3,"c"]]

say(list_group(ns, x -> x % 2)) # {1:[3,1,1,5,9], 0:[4,2,6]}

say(list_find(ns, x -> x > 7)) # 9
```

use io

Чтение и запись файлов.

Функция	Описание
`file_create(path)`	Создать пустой файл (если уже есть — не трогает)
`file_read(path)`	Читать весь файл в строку (ошибка если нет)
`file_write(path, text)`	Записать строку в файл (перезаписать)
`file_append(path, text)`	Добавить строку в конец файла
`file_lines(path)`	Читать файл, вернуть список строк (ошибка если нет)
`file_exists(path)`	Проверить существование файла
`file_delete(path)`	Удалить файл

```
use io

# Создать файл

file_create("log.txt")

say(file_exists("log.txt")) # true

# Записать и читать

file_write("log.txt", "старт\n")

file_append("log.txt", "шаг 1\n")

file_append("log.txt", "шаг 2\n")

say(file_read("log.txt"))
```

```
say(file_lines("log.txt")) # ["старт", "шаг 1", "шаг 2"]

# Удалить
file_delete("log.txt")
say(file_exists("log.txt")) # false
```

use os

Системные вызовы, переменные окружения, время.

Функция	Описание	Пример
os_env(name)	Значение переменной окружения	os_env("HOME") → "/root"
os_run(cmd)	Выполнить команду (код возврата)	os_run("mkdir tmp")
os_shell(cmd)	Выполнить и получить вывод	os_shell("echo hi") → "hi"
os_exit(code)	Завершить программу	os_exit(0)
os_time()	Unix-время в секундах	os_time() → 1774213685
os_clock()	Время CPU в секундах	os_clock() → 0.002

```
use os
say("HOME =", os_env("HOME"))
say("время =", os_time())

set out = os_shell("uname -s")
say("ОС =", out)

# Замерить время выполнения
set t1 = os_clock()
set s = 0
loop for i in range(100000):
    s += i
end
```

```
set t2 = os_clock()

say("sum =", s)

say("время =", t2 - t1, "сек")
```

22. ООП: obj / extends / self / super

KITE поддерживает объектно-ориентированное программирование через ключевое слово **obj**. Классы содержат поля (**set**) и методы (**def**). Первый параметр каждого метода — self (ссылка на текущий объект). Создание объекта: **ClassName.new**(...). Наследование: **extends**.

Объявление класса:

```
obj Animal:
    set name = nil # поле по умолчанию nil

    def init(self, name):
        self.name = name
    end

    def speak(self):
        say("${self.name} издаёт звук")
    end

    def str(self):
        give "<Animal: ${self.name}>"
    end
end

set a = Animal.new("Кот")
a.speak() # Кот издаёт звук
say(a.name) # Кот
say(type(a)) # Animal
say(a is Animal) # true
```

Наследование:

```
obj Dog extends Animal:
    set tricks = 0

    def init(self, name):
        self.name = name
        self.tricks = 0
    end

    def learn(self):
        self.tricks += 1
        say("${self.name} выучил трюк #${self.tricks}")
    end

    def speak(self): # переопределение метода
        say("${self.name}: Гав! (трюков: ${self.tricks})")
    end
end

set d = Dog.new("Шарик")
d.speak()
d.learn()
d.learn()
say(d is Dog) # true
say(d is Animal) # true — наследование работает
```

super — вызов родительского метода:

```
obj Cat extends Animal:
    def speak(self):
```

```
        super.speak(self) # сначала Animal.speak
        say("${self.name}: Мяу!")
    end
end

obj KittyCat extends Cat:
    def speak(self):
        super.speak(self) # сначала Cat.speak (который вызовет Animal.speak)
        say("(тихо мурчит)")
    end
end

set k = KittyCat.new("Пушок")
k.speak()
# Пушок издаёт звук
# Пушок: Мяу!
# (тихо мурчит)
```

Ключевые слова и операции ООП:

Синтаксис	Описание
`obj Name:`	Объявить класс
`obj Child extends Base:`	Наследовать от Base
`set field = nil`	Объявить поле с дефолтом nil
`def init(self, ...):`	Конструктор — вызывается при .new()
`self.field`	Обратиться к полю объекта
`self.method()`	Вызвать метод объекта
`super.method(self)`	Вызвать метод родительского класса
`ClassName.new(...)`	Создать новый объект
`obj is ClassName`	Проверить тип объекта

Синтаксис	Описание
`type(obj)`	Вернуть имя класса строкой

23. Инкапсуляция и полиморфизм

Инкапсуляция — приватные поля и методы:

Поля и методы с префиксом _ являются приватными. К ним можно обращаться только внутри методов того же класса или его наследников. Попытка обратиться снаружи вызывает `AccessError`.

```
obj BankAccount:
    set _balance = 0 # приватное поле
    set owner = nil # публичное поле

    def init(self, owner, start):
        self.owner = owner
        self._balance = start
    end

    def deposit(self, n):
        when n <= 0: say("Сумма должна быть > 0") give nil end
        self._balance += n
        say("Пополнено +${n}. Баланс: ${self._balance}")
    end

    def withdraw(self, n):
        when n > self._balance:
            say("Недостаточно средств!")
            give nil
        end
        self._balance -= n
    end
```

```
    def balance(self):
        give self._balance # даём доступ через метод
    end
end

set acc = BankAccount.new("Алиса", 1000)
acc.deposit(500)
say(acc.balance()) # 1500 — OK через метод
say(acc.owner) # Алиса — публичное поле

# Прямой доступ к приватному — ошибка:
do:
    say(acc._balance)
err AccessError:
    say("Ошибка: ${err_msg}")
end
```

Полиморфизм:

Полиморфизм позволяет работать с объектами разных классов через единый интерфейс. Каждый класс реализует метод по-своему, но вызывается одинаково.

```
use math

obj Shape:
    set _color = "чёрный"

    def init(self, color):
        self._color = color
    end
```

```
    def area(self): # базовый метод
        give 0
    end

    def describe(self): # использует полиморфный area()
        say("${self._color} фигура, площадь = ${self.area()}")
    end
end

obj Circle extends Shape:
    set _r = 0
    def init(self, color, r):
        self._color = color
        self._r = r
    end
    def area(self): # переопределение
        give PI * self._r * self._r
    end
end

obj Rectangle extends Shape:
    set _w = 0
    set _h = 0
    def init(self, color, w, h):
        self._color = color
        self._w = w
        self._h = h
    end
```

```
    def area(self): # переопределение
        give self._w * self._h
    end
end

# Полиморфный код — не знает конкретный тип:
set shapes = [
    Circle.new("красный", 5),
    Rectangle.new("синий", 4, 6),
    Circle.new("зелёный", 3)
]

set total = 0
loop for s in shapes:
    s.describe() # каждый вызывает свой area()
    total += s.area()
end
say("Общая площадь:", round(total))
```

Правила приватности: поле или метод с _ в начале имени доступны только внутри методов класса и его наследников. При нарушении выбрасывается `AccessError` которую можно поймать через **do** / **err** `AccessError`.

KITE

Минимальный язык. Максимальные возможности.

Написан на C · Работает в Termux · v1.2